第三阶段：NEMU 扩展与协同仿真

工大喵喵队 2026 年 11 月

一、NEMU 改动

NEMU 里加 vdot 的译码和 helper，让它能算出同样的点积结果，再和香山 difftest 逐条比对。

译码在 `src/isa/riscv64/instr/decode.c` 加一行：

```
def_INSTR_IDTAB("0000001 ????? ????? 010 ????? 00010 11", OP_V, vdot);
```

指令表里加：

```
#define VECTOR_INSTR_TERNARY(f) \
  f(vdot) f(vadd) f(vsub) f(vrsub) f(vminu) f(vmin) \
  ...
```

helper：

```
// eight low int8 pairs, int32 result, zero upper 96 bits.
// read all sources before writing (vd may alias vs1/vs2/v0).
def_EHelper(vdot) {
  require_vector(true);
  /* decode-stage: keep instruction word as mtval */
  if (vtype->vsew != 0 || vtype->vlmul != 0 ||
      vstart->val != 0 || s->vm != 1) {
    longjmp_exception(EX_II);
  }
  /* execute-stage: VL mismatch, mtval=0 */
  if (vl->val != 8) {
    vdot_illegal_no_tval = 1;
    longjmp_exception(EX_II);
  }
  int32_t sum = 0;
  for (int i = 0; i < 8; i++) {
    int32_t a = vreg_b(id_src->reg, i);
    int32_t b = vreg_b(id_src2->reg, i);
    if (a >= 128) a -= 256;
    if (b >= 128) b -= 256;
    sum += a * b;
  }
  set_vreg(id_dest->reg, 0, (uint32_t)sum, 3, 0, 0);
  set_vreg(id_dest->reg, 1, 0, 3, 0, 0);
  vstart->val = 0;
  vp_set_dirty();
}
```

几个检查：

检查/动作	说明
require_vector(true)	检查向量状态合法
vtype->vsew != 0	SEW 必须是 8（译码阶段，mtval 保留指令字）
vtype->vlmul != 0	LMUL 必须是 1（译码阶段）
vstart->val != 0	不支持非零起始（译码阶段）
s->vm != 1	不支持掩码（译码阶段）
vl->val != 8	VL 必须是 8（执行阶段，mtval=0）
vreg_b	按字节读源，读 8 次
set_vreg(..., 0, ...)	低 32 位写结果
set_vreg(..., 1, 0, ...)	高 96 位清零
vstart->val = 0	完成后清 vstart
vp_set_dirty()	标记向量寄存器脏

先读后写很重要：如果 vd 和 vs1/vs2 重叠，先把 8 对源字节读进局部变量，再写 vd，避免读到自己刚写的结果。

二、异常对齐

1.mtval

RTL 的 mtval 分两类：

· VL 类（VL=0/1/7/9）在执行阶段由 VIMacU dotIllegal 发现，没有 trap instruction 记录，mtval=0；

· SEW/LMUL/vstart 类在译码阶段由 DecodeUnit 发现，TrapInstMod 把指令字填进 mtval，实测是 0x0220a18b。

旧 NEMU helper 把所有这些情况都当成“vdot 非法，tval=0”，所以 SEW=16、LMUL=2、vstart=1 三种配置和 RTL 不一致。修复是把条件拆成两段：

```
/* 译码阶段：SEW/LMUL/vstart/vm 不匹配，保留指令字 mtval */
if (vtype->vsew != 0 || vtype->vlmul != 0 || vstart->val != 0 || s->vm != 1) {
  longjmp_exception(EX_II);
}
/* 执行阶段：VL != 8，mtval=0 */
if (vl->val != 8) {
  vdot_illegal_no_tval = 1;
  longjmp_exception(EX_II);
}
```

intr.c 把全局标志改成 EX_II/EX_VI 分支内的快照，再决定 mtval 是 0 还是 cpu.instr：

```
case EX_II: case EX_VI:
  int vdot_no_tval = vdot_illegal_no_tval;
  vdot_illegal_no_tval = 0;
  mtval->val = vdot_no_tval ? 0 : MUXDEF(CONFIG_TVAL_EX_II, cpu.instr, 0);
  break;
```

A/B 对照（日志在 results/t5-nemu-fixed-regressions/）：

· 修复前 sew16-diff-old-so.log：mtval different ... right=0, wrong=0x0220a18b，说明 difftest 确实比对 mtval，旧模型确实错；

· 修复后 sew16-diff-new-so.log：同一用例 HIT GOOD TRAP，无 mismatch；

· vl0-diff-new-so.log：VL=0 一侧无 mismatch。

A/B 只覆盖 SEW=16 和 VL=0；LMUL=2、vstart=1 的 mtval 由 probe 逐例测量，还没有带 difftest 的 A/B。S/VS 委托、RNMI、NMIE=0 早退路径没覆盖，所以文档只写“已测的 M 模式配置一致”，不写“所有异常路径完全对齐”。

2.NMIE/MDT

香山 NewCSR/MachineLevel.scala 里 NMIE 复位值是 1；NEMU 默认没有开 CONFIG_NMIE_INIT，init.c 会把 mnstatus.nmie 设成 0。结果非法指令异常进 raise_intr 时，if (!mnstatus->nmie)提前返回，mcause/mepc 不写，和 RTL 对不上。

配置：

```
CONFIG_RV_SMRNMI=y
CONFIG_NMIE_INIT=y
# CONFIG_MDT_INIT is not set
CONFIG_TVAL_EX_II=y
```

· NMIE_INIT=y：复位 NMIE=1，和 RTL 对齐；

· MDT_INIT 关闭：复位 MDT=0，避免 M 模式异常走 double-trap；

· TVAL_EX_II=y：标准非法指令仍然保留指令字 tval，vdot 的零 tval 标志只用于 VL 类执行阶段异常。

三、difftest 流程

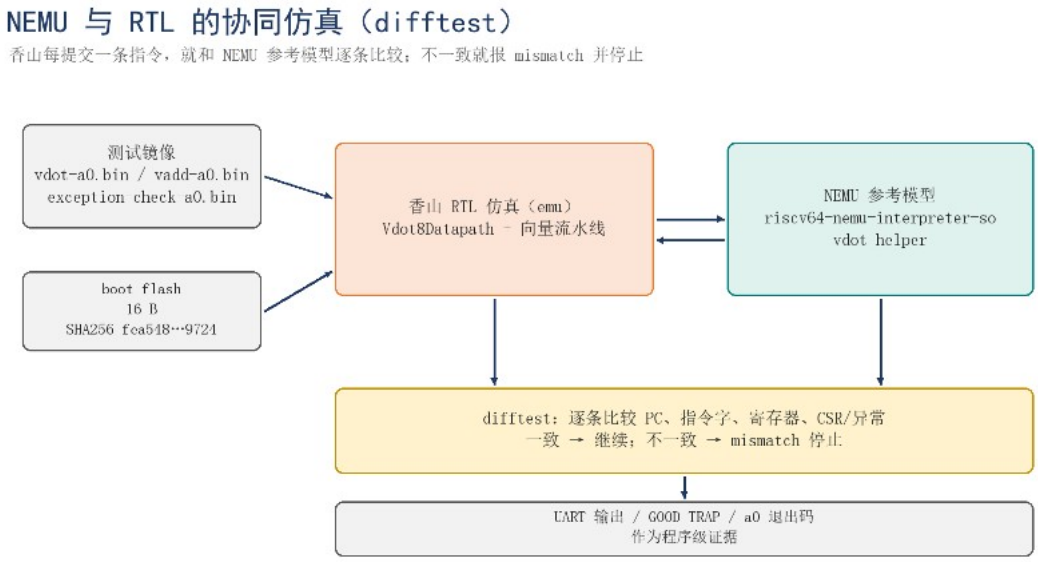


图 1 NEMU 与 RTL 的 difftest 结构

启动命令：

```
cd $NOOP_HOME
./build/emu \
  -i $HOME/risv/project/software/build/vdot-a0.bin \
  --diff $NEMU_HOME/build/riscv64-nemu-interpreter-so \
  -F $HOME/risv/artifacts/boot-minimal.bin
```

· -i：测试程序；

· --diff：加载 NEMU 共享库，开 difftest；

· -F：16 字节 boot flash，两边必须用同一个文件。

日志开头会打印：

```
Using simulated 32768B flash
The image is .../software/build/vdot-a0.bin
The reference model is .../riscv64-nemu-interpreter-so
The first instruction of core 0 has commited. Difftest enabled.
```

香山每提交一条指令，difftest 就把 PC、指令字、寄存器写回、CSR/异常状态和 NEMU 比一遍。一致就继续，不一致就打印 mismatch 并终止。

日志里有很多[PERF]行，那是性能计数器，不是 difftest 结果。判断运行是否结束看：

- HIT GOOD TRAP at pc = 0x80000032;
- Core-0 instrCnt = ... cycleCnt = ...;
- 没有 mismatch、CRITICAL ERROR、断言失败。

要判断 a0=0，要看 a0 版 trap 字（.word 0x0005006b）的 GOOD TRAP，不能只看历史的 x0 版 HIT GOOD TRAP。

四、三项 a0 回归

1.vdot 功能

```
Core 0: HIT GOOD TRAP at pc = 0x80000032
Core-0 instrCnt = 841,260, cycleCnt = 790,553, IPC = 1.064141
```

results/t5b-clean-ref/vdot-a0.log，1,030 个程序级用例 × 5 个实现变体，包含正数、负数、边界、随机、别名、v0，a0=0（退出码 0），无 mismatch。

2.vadd 标准回归

```
Core 0: HIT GOOD TRAP at pc = 0x80000032
Core-0 instrCnt = 23,876, cycleCnt = 16,870, IPC = 1.415293
```

results/t5b-clean-ref/vadd-a0.log，a0=0，用来确认 vdot 补丁没有破坏原来的向量路径。

3.VL=0 异常

```
Core 0: HIT GOOD TRAP at pc = 0x80000032
Core-0 instrCnt = 274, cycleCnt = 2,865, IPC = 0.095637
```

results/t5b-clean-ref/exception-check-a0.log，a0=0。三次 trap 都符合预期：mcause=2、mepc 指向 vdot、mtval=0、v3 保持 0xFF、标准非法指令

mtval=0x7B 没被污染、第二次 VL=0 还能 trap。8 种非法配置的 mtval 见 results/t2-mtval-probe/mtval-probe.log。

五、遇到的问题

现象	原因	处理
第 2 条指令 mismatch	emu 和 NEMU 默认 flash 不同	统一用 -F 传同一个 16 B flash
异常 mcause/mepc=0	NMIE 复位值不同，raise_intr 提前返回	CONFIG_NMIE_INIT=y
仍然不一致	MDT 复位/路径	关闭 CONFIG_MDT_INIT
mtval 不同	NEMU 把所有 vdot 非法都写成 0，RTL 译码类要写指令字	拆成译码/执行两段，只有 VL 类置 0
日志交错	NEMU 诊断输出和 UART 抢终端	以完整行 GOOD TRAP、PERF_V1 为准

基本是按“先 flash、再 NMIE/MDT、再 mtval 分阶段”的顺序解决的。

六、覆盖与限制

覆盖：

· NEMU 译码、helper、写回、vstart、dirty；

· RTL/NEMU 逐指令比对；

· 正/负/边界/随机/别名/v0；

· M 模式 8 种非法配置的 mtval（VL 类 4 种、SEW/LMUL/vstart3 种、标准非法 1 种），由 probe 逐例打印；

· 标准 vadd 回归。

没覆盖：

· S/VS 委托、RNMI、NMIE=0 早退路径；

· vm=0 等其他非法配置；

· 重新初始化后的零 tval 标志状态；

· 随机指令流和覆盖率统计；

· 参数化只到独立模块级，和 NEMU 协同仿真无关。